PROJECT DOCUMENTATION

Mini Freelancing Marketplace

A Web-Based Platform for Clients and Freelancers

Student Name	Shreya Gupta
Project Type	Full-Stack Web Application
Technology Stack	PHP, Laravel, MySQL, HTML5, CSS3, JavaScript
Development Environment	Visual Studio Code, Composer, Laravel, MySQL / XAMPP
Project Level	Beginner to Intermediate
Planned Development Duration	14 Days (MVP Scope)
AF Code	AF05323537

Prepared for academic and portfolio project planning

TABLE OF CONTENTS

Section	Page
1. Project Overview	3
2. System Requirements	4
3. Technology Stack	4
4. System Architecture	5
5. Database Design	6
6. Module Description	7
7. Suggested Laravel Folder Structure	8
8. Development Methodology	8
9. Validation, Security, and Exception Handling	9
10. Testing	10
11. Installation and Execution	11
12. Limitations	12
13. Future Enhancements	12
14. Conclusion	12
15. References	12

1. Project Overview

1.1 Project Title

Mini Freelancing Marketplace

1.2 Introduction

The Mini Freelancing Marketplace is a web-based application designed to connect clients who need work completed with freelancers who can provide services. Clients can publish projects with descriptions, budgets, deadlines, and required skills. Freelancers can browse available projects and submit proposals containing their approach, proposed price, and estimated delivery time. Clients can review proposals and select a suitable freelancer.

The application is planned using PHP and Laravel for server-side development, MySQL for data storage, and HTML, CSS, and JavaScript for the user interface. The initial version focuses on the core marketplace workflow and is intended to be achievable within a two-week development schedule.

1.3 Problem Statement

Finding suitable freelance work and managing small projects through scattered messages can make it difficult to compare offers, track applications, and keep project information organized. Clients may receive proposals in different formats, while freelancers may struggle to discover relevant opportunities. The proposed system provides one organized platform for project posting, proposal submission, selection, and basic progress tracking.

1.4 Objectives

- Provide a simple and user-friendly platform for clients and freelancers.
- Allow users to register, log in, and access features according to their role.
- Enable clients to create, view, edit, and manage project listings.
- Enable freelancers to search for projects and submit proposals.
- Allow clients to review proposals and accept or reject applications.
- Track project status from open to in progress and completed.
- Store user, project, and proposal information in a relational MySQL database.
- Apply validation, authentication, and authorization to protect data.

1.5 Project Scope

The first version includes user registration and login, client and freelancer roles, profile details, project management, project browsing and filtering, proposal submission, proposal review, and basic project status tracking. The two-week MVP does not require online payments or a real-money escrow system.

Possible future additions include reviews and ratings, notifications, file delivery, advanced skill matching, an administrator dashboard, payment integration, and reporting.

> **In simple terms:** the first version focuses on the main journey—clients post projects, freelancers apply, clients choose a proposal, and both sides track progress. Extra features can be added after this workflow works reliably.

2. System Requirements

2.1 Functional Requirements

Module	Description
User Registration	Allows new users to create an account with name, email, password, and role.
User Login and Logout	Authenticates registered users and ends their session securely.
Profile Management	Allows users to maintain a short bio, skills, and portfolio information.
Project Posting	Allows clients to publish projects with title, description, category, budget, skills, and deadline.
Project Browsing	Allows freelancers to view open projects and search or filter listings.
Proposal Submission	Allows freelancers to submit a cover letter, bid amount, and estimated delivery time.
Proposal Review	Allows the project-owning client to view, accept, or reject proposals.
Project Tracking	Displays project status such as Open, In Progress, Submitted, or Completed.
Dashboard	Shows relevant projects and proposals for the signed-in user.

2.2 Non-Functional Requirements

- Security: Passwords must be hashed and restricted pages protected through authentication and authorization.
- Usability: Navigation, forms, and status labels should be clear and consistent.
- Performance: Common pages and project listings should load within a reasonable time.
- Reliability: Project and proposal updates should be saved accurately.
- Maintainability: Use Laravel's organized MVC structure and reusable components.
- Compatibility: Support modern desktop and mobile browsers with responsive layouts.
- Data Integrity: Validate required fields, budgets, deadlines, and status changes on the server.

3. Technology Stack

Technology	Purpose
HTML5	Defines page structure, forms, tables, and navigation.
CSS3	Styles the interface and provides responsive layouts.
JavaScript	Adds client-side interactivity and form feedback.

PHP	Runs server-side application logic.
Laravel	Provides routing, controllers, models, validation, middleware, and application structure.
MySQL	Stores user accounts, profiles, projects, proposals, and related records.
Composer	Installs and manages PHP dependencies.
Visual Studio Code	Code editor for development and debugging.
XAMPP / MySQL service	Can provide a local MySQL database and local development environment.
Web Browser	Allows users to access and test the application.

3.1 Development Approach

The recommended approach is Laravel Blade with HTML, CSS, and JavaScript. This keeps the frontend and backend in one Laravel application and is more manageable for an individual project with a two-week deadline. A separate frontend framework is not required for the MVP.

4. System Architecture

The Mini Freelancing Marketplace follows a three-layer architecture: Presentation Layer, Application Layer, and Database Layer.

4.1 Presentation Layer

This layer contains browser pages built with Blade templates, HTML, CSS, and JavaScript. It includes registration and login pages, project listings, project details, proposal forms, and user dashboards.

4.2 Application Layer

Laravel routes incoming requests to controllers. Controllers validate input and coordinate actions such as creating projects, submitting proposals, accepting applications, and changing project status. Middleware and authorization checks restrict access to the appropriate user.

4.3 Database Layer

MySQL stores persistent records. Laravel models and Eloquent relationships connect users, profiles, projects, proposals, and optional review records.

4.4 Request Flow

1. A user opens a page in a web browser.
2. The user submits an action, such as posting a project or applying for a project.
3. Laravel routes the request to the relevant controller.
4. The application validates the input and checks authentication, ownership, and permissions.
5. Laravel reads or updates the relevant MySQL records through models.
6. The application returns a success message, validation error, or updated page.

5. Database Design

The database is designed around the relationships between users, projects, and proposals. A user can post multiple projects as a client and submit multiple proposals as a freelancer. Each proposal belongs to one project and one freelancer.

5.1 Database Name

Suggested database name: mini_freelance_marketplace

5.2 Main Tables

Table	Important Columns	Purpose
users	id, name, email, password, role, created_at, updated_at	Stores login and role information. Passwords are stored as hashes.
freelancer_profiles	id, user_id, bio, skills, portfolio_url, created_at, updated_at	Stores additional freelancer information.
categories	id, name, description	Groups projects into categories.
projects	id, client_id, category_id, title, description, budget, deadline, status, timestamps	Stores client project listings.
proposals	id, project_id, freelancer_id, cover_letter, bid_amount, delivery_days, status, timestamps	Stores applications submitted by freelancers.
reviews (future)	id, project_id, reviewer_id, freelancer_id, rating, comment	Stores feedback after project completion.

5.3 Relationships

- Users to Projects: One client can create many projects.
- Users to Proposals: One freelancer can submit many proposals.
- Projects to Proposals: One project can receive multiple proposals.
- Categories to Projects: One category can contain multiple projects.
- Projects to Reviews: A completed project may have a review in a future version.
- Primary keys uniquely identify records; foreign keys link related records.

5.4 Suggested Status Values

Record	Suggested Values	Meaning
Project	open, in_progress, submitted, completed, cancelled	Tracks the project lifecycle.
Proposal	pending, accepted, rejected, withdrawn	Tracks the client's decision and application state.

5.5 Database Integrity Rules

- Email addresses should be unique.
- Project budgets must be valid non-negative values, with the minimum business rule defined by the application.
- A freelancer should not submit duplicate proposals for the same project.
- Only the client who owns a project can manage its proposals.
- Only one proposal should be accepted for a project in the basic version.
- Accepting a proposal and updating project status should be handled consistently, preferably in a database transaction.

6. Module Description

6.1 User Registration and Login

Users create accounts with a name, email, password, and role. Login verifies credentials and directs users to the appropriate dashboard. Passwords must be hashed using Laravel's hashing facilities. Protected routes must require authentication.

6.2 Profile Management

Freelancers can provide a short biography, list their skills, and add a portfolio URL. Client profiles may contain basic contact or organization information if required. Users should only be able to update their own profiles.

6.3 Project Management

Clients can create a project by entering its title, description, category, budget, deadline, and required skills. The client can edit or close their own project. Server-side validation should check required fields and prevent invalid deadlines or budgets.

6.4 Project Search and Filtering

Freelancers can browse open projects, search by title or keyword, filter by category, and optionally sort by budget or latest posting. Search results should be paginated to keep pages manageable.

6.5 Proposal Management

A freelancer submits a cover letter, proposed price, and delivery estimate for an open project. The client who owns that project can view the proposals and accept or reject them. The application must prevent a user from applying to a closed project or changing another user's proposal.

6.6 Project Tracking

When a proposal is accepted, the project status changes to In Progress. The MVP can allow the client or assigned freelancer to move the project to a submitted or completed state according to the defined workflow. Every status change must be checked against the user's permissions.

6.7 Dashboard

The client dashboard can show posted projects and received proposals. The freelancer dashboard can show submitted proposals, accepted projects, and completed projects. Counts should be calculated from database records rather than hard-coded values.

7. Suggested Laravel Folder Structure

```
mini-freelance-marketplace/
├── app/
│   ├── Http/Controllers/
│   │   ├── ProjectController.php
│   │   ├── ProposalController.php
│   │   └── ProfileController.php
│   ├── Models/
│   │   ├── User.php
│   │   ├── Project.php
│   │   ├── Proposal.php
│   │   ├── Category.php
│   │   └── FreelancerProfile.php
│   └── Policies/
├── database/
│   ├── migrations/
│   └── seeders/
├── resources/views/
│   ├── layouts/
│   ├── auth/
│   ├── dashboard/
│   ├── projects/
│   ├── proposals/
│   └── profile/
├── public/
│   ├── css/
│   ├── js/
│   └── images/
├── routes/
│   └── web.php
├── .env
├── composer.json
└── README.md
```

This structure follows Laravel's MVC pattern. Routes define URLs, controllers process requests, models interact with database records, Blade views render pages, migrations define database tables, and policies or middleware enforce access control.

8. Development Methodology

1. Requirement Analysis: Identify the roles, core user journeys, and features required for the two-week MVP.
2. System Design: Plan the page layout, database tables, relationships, and project/proposal status flow.
3. Environment Setup: Install or verify PHP, Composer, Laravel, and MySQL; create the project and configure the environment.
4. Database Development: Create migrations and models for users, profiles, categories, projects, and proposals.
5. Frontend Development: Build responsive Blade pages, forms, navigation, listings, and dashboards.
6. Backend Development: Implement authentication, project CRUD, search, proposal submission, and proposal decisions.
7. Validation and Security: Add server-side validation, authorization checks, and safe file and data handling where applicable.
8. Testing: Run functional tests for both client and freelancer roles and fix discovered defects.
9. Finalization: Prepare the README, screenshots, setup instructions, and a demonstration workflow.

8.1 Two-Week Development Schedule

Days	Planned Work	Deliverable
1–2	Project setup, requirements, database design, initial UI	Laravel app and database plan
3–4	Authentication, roles, profile basics	Users can register and log in
5–6	Project create/read/update/delete, project list	Clients can manage projects
7–9	Proposal form, proposal list, accept/reject flow	Core application workflow
10–11	Dashboards, search/filtering, status tracking	Usable role-specific dashboards
12–14	Testing, bug fixes, responsive styling, README	Demonstrable MVP

This schedule is an estimate. If Laravel is new to the developer, the scope should be reduced before removing time reserved for testing. Payment processing, advanced notifications, and AI features are outside the initial two-week target.

9. Validation, Security, and Exception Handling

9.1 Input Validation

- Name must not be empty and email must have a valid format.
- Email must be unique when a user registers.
- Password must meet the application's security rules and must never be stored as plain text.
- Project title, description, category, budget, and deadline must be validated.
- Proposal bid amount and delivery estimate must contain valid values.
- A freelancer cannot submit a proposal more than once for the same project if duplicate proposals are disallowed.
- Users can only edit projects, proposals, and profiles they are authorized to manage.
- A proposal cannot be accepted if the project is no longer open or another proposal has already been accepted.

9.2 Security Practices

- Use Laravel authentication and middleware to protect restricted pages.
- Use authorization policies or equivalent ownership checks for project and proposal actions.
- Use Laravel validation and Eloquent or parameterized database operations instead of building SQL from untrusted input.
- Use CSRF protection for web forms.
- Display friendly error messages and log technical details securely rather than exposing stack traces.
- Use database transactions for related changes that must succeed together.

9.3 Error Handling

When an operation fails, the application should preserve user input where appropriate, display a clear error message, and log details for debugging. For example, if a proposal cannot be accepted because the project is already assigned, the application should reject the action and explain that the project is no longer available.

10. Testing

Testing checks that the marketplace behaves correctly for clients and freelancers and that unauthorized actions are prevented.

10.1 Test Cases

Test ID	Test Scenario	Expected Result
TC01	Register with valid details	Account is created successfully.
TC02	Register with an email already in use	Validation error is displayed.
TC03	Log in with invalid credentials	Access is denied and an error is shown.
TC04	Client creates a valid project	Project is saved and appears in the listing.
TC05	Submit a project form with missing required fields	Validation messages are displayed.
TC06	Freelancer searches for an open project	Matching projects are shown.
TC07	Freelancer submits a valid proposal	Proposal is saved and visible to the project owner.
TC08	Freelancer attempts a duplicate proposal	Duplicate submission is rejected if disallowed.
TC09	A different client attempts to manage a project	Unauthorized action is prevented.
TC10	Client accepts one proposal	Proposal is accepted and project status updates consistently.
TC11	Try to apply to a closed project	Proposal submission is rejected.
TC12	User opens a protected page while logged out	User is redirected to login or access is denied.
TC13	Freelancer views their dashboard	Only their own proposals and assigned projects are shown.
TC14	User logs out	Authenticated session ends.

10.2 Types of Testing

- Functional Testing: Verifies project posting, browsing, proposals, and status changes.
- Integration Testing: Checks interactions between Laravel controllers, models, and MySQL.
- Validation Testing: Confirms invalid input is rejected with useful messages.
- Security Testing: Checks authentication, authorization, CSRF protection, and unsafe input handling.
- Responsive Testing: Checks usability on desktop and mobile-sized screens.
- User Acceptance Testing: Confirms that the main client and freelancer workflows meet project requirements.

11. Installation and Execution

11.1 Prerequisites

- A compatible PHP version supported by the chosen Laravel release.
- Composer installed and available in the terminal.
- Laravel project dependencies.
- MySQL server or a local environment such as XAMPP.
- Visual Studio Code or another code editor.
- A modern web browser.

11.2 Setup Steps

1. Create a Laravel project using the Composer/Laravel installation method appropriate for the installed versions.
2. Open the project directory in Visual Studio Code.
3. Create a MySQL database named mini_freelance_marketplace.
4. Configure the database connection in the project's .env file, including DB_DATABASE, DB_USERNAME, and DB_PASSWORD.
5. Install dependencies if needed by running composer install.
6. Generate the application key with php artisan key:generate if the project requires it.
7. Create the required migration files and run php artisan migrate.
8. Configure and test the authentication flow and role handling.
9. Start the development server using php artisan serve.
10. Open the local URL printed by Laravel in the browser and test the client and freelancer workflows.

Exact commands can vary by Laravel version and the way the project is created. Database credentials should be configured in the development environment and must not be committed to a public repository.

12. Limitations

- The MVP does not include real-money payment processing or escrow.
- Email and SMS notifications may not be included in the two-week version.
- Advanced dispute resolution and contract management are outside the initial scope.
- Reviews and ratings may be deferred until project completion workflows are stable.
- The initial skill-matching feature, if added, will use simple rule-based matching rather than advanced AI.
- Production deployment requires additional configuration, monitoring, backups, and security review.

13. Future Enhancements

1. Add reviews and ratings for completed projects.
2. Add in-app and email notifications for proposal decisions and status updates.
3. Add secure file delivery for project requirements and completed work.
4. Add skill-based matching between freelancer profiles and project requirements.
5. Add an administrator dashboard for managing users and reported content.
6. Add richer freelancer portfolios and work samples.
7. Add project milestones and progress percentages.
8. Integrate a payment provider only after defining security, legal, and operational requirements.
9. Add analytics for project activity and proposal conversion.
10. Improve accessibility, performance, logging, and automated test coverage.

14. Conclusion

The Mini Freelancing Marketplace demonstrates how PHP, Laravel, MySQL, HTML, CSS, and JavaScript can be combined to develop a practical web application. The system brings clients and freelancers together through project listings, proposal submission, application decisions, and basic progress tracking.

The project provides experience with Laravel's MVC structure, relational database design, CRUD operations, authentication, authorization, form validation, and testing. By prioritizing the core workflow, an individual developer can aim to deliver a functional MVP in two weeks and expand it later with reviews, notifications, file delivery, and other enhancements.

15. References

- PHP Official Documentation — https://www.php.net/docs.php
- Laravel Documentation — https://laravel.com/docs
- MySQL Official Documentation — https://dev.mysql.com/doc/
- MDN Web Docs — https://developer.mozilla.org/
- OWASP Web Application Security Guidance — https://owasp.org/